# ✈️ Big Data Project

End-to-End Airline Data Pipeline on AWS

# 📋 Agenda

1. **Project Overview & Goals**
2. **Architecture**
3. **Infrastructure as Code (Terraform)**
4. **Data Lake Design (S3)**
5. **ETL Pipeline (PySpark on EMR)**
6. **Orchestration (Apache Airflow)**
7. **Data Catalog & Query Layer (Glue + Athena)**
8. **DevOps & Automation**
9. **Demo Flow**
10. **Lessons Learned & Next Steps**

# 🎯 Project Overview

**Goal:** Build a scalable batch data pipeline to ingest, transform, and analyze US airline flight data from Kaggle.

## Key Objectives

- Ingest raw CSV data from Kaggle → S3 data lake
- Transform and curate data using **PySpark on EMR**
- Catalog processed data with **AWS Glue**
- Enable ad-hoc queries via **Amazon Athena**
- Orchestrate the full workflow with **Apache Airflow**
- Provision all infrastructure with **Terraform**

# 🏗️ Architecture

```
┌──────────────────┐
│   Airflow        │
│  (Docker on      │
│   EC2)           │
└──────────────────┘
        │
        ▼
┌──────────────────┐
│     EMR          │
│  (PySpark)       │
│  1M + 2C + 1T    │
└──────────────────┘
```

```
┌──────────────────────────────────────┐
│            S3 Data Lake              │
│   /raw/          ← source CSVs       │
│  /scripts/     ← PySpark ETL         │
│  /processed/   ← Parquet out         │
│ /athena-results/                     │
└──────────────────────────────────────┘
```

```
reads/
writes
```

```
┌──────────────────┐
│  Glue Crawler    │
│  + Data Catalog  │
└──────────────────┘
        ▼
┌──────────────────┐
│     Athena       │
└──────────────────┘
```

▶ Tableau

# 🧱 Infrastructure as Code — Terraform

## Modular Design (6 Modules)

| Module | Resources |
|---|---|
| `s3` | Versioned bucket, public access blocked, prefixes |
| `iam` | 5 IAM roles (EC2, EMR Service, EMR EC2, Glue, Athena) |
| `ec2_airflow` | EC2 instance ( `t3.micro` ), security group, CloudWatch |
| `emr` | Cluster (1 Master + 2 Core + 1 Task), security groups |
| `glue` | Crawler, Catalog Database, CloudWatch logs |
| `athena` | Workgroup, named sample query |

```
terraform init → plan → apply    # One command deploys everything
```

# 🧱 Terraform — Provider & Backend

```
terraform {
  required_version = "~> 1.5"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  # S3 remote backend with DynamoDB locking (optional)
}

provider "aws" {
  region = var.aws_region          # default: us-east-1
  default_tags { tags = var.tags }
}
```

- Uses **default VPC & subnets** — no custom networking needed
- All resources tagged: `Project`, `ManagedBy`, `Environment`

# 🪣 Data Lake Design — S3

## Bucket Structure

```
s3://bigdata-project-data-lake/
├── raw/
│   └── airline_data/           ← Raw CSVs from Kaggle
│       └── *.csv
├── scripts/
│   ├── ingest.py               ← PySpark ingest job
│   ├── merge.py                ← Holiday merge job
│   └── aggregate.py            ← Aggregation job
├── processed/
│   ├── airline_data/           ← Curated Parquet (partitioned)
│   └── question_1/
│       ├── merged/             ← Holiday-merged output
│       └── aggregated/         ← Final aggregation
├── athena-results/             ← Athena query output
└── emr-logs/                   ← EMR cluster logs
```

# ⚡ ETL — PySpark Ingest Job

`scripts/ingest.py`

**Input:** Raw CSV files from `s3://.../raw/airline_data/`

**Transformations:**

1. Read with explicit schema (29 raw columns)
2. Standardize column names → `snake_case`
3. Cast to proper types (integers, strings)
4. Uppercase carrier codes, airports
5. Add computed fields:
    - `flight_date` — derived date column
    - `dep_hour`, `arr_hour` — time bucketing
    - `route` — `origin → dest`

# ⚡ ETL — Multi-Stage Pipeline

## Three PySpark Jobs on EMR

| Stage | Script | Purpose |
|---|---|---|
| **Ingest** | `ingest.py` | Raw CSV → curated Parquet |
| **Merge** | `merge.py` | Join flight data with holiday reference |
| **Aggregate** | `aggregate.py` | Compute delay metrics by holiday proximity |

## EMR Cluster Configuration

- **Release:** `emr-6.15.0` with Spark
- **Topology:** 1 Master + 2 Core + 1 Task (`m5.xlarge` each)
- **Auto-termination:** 1 hour idle timeout
- **Deploy mode:** `cluster` for fault tolerance

# 🔄 Orchestration — Apache Airflow

## Dockerised Deployment

| Service | Description |
|---|---|
| `postgres` | Metadata DB (PostgreSQL 16) |
| `airflow-init` | DB migration + admin user creation |
| `airflow-webserver` | UI on port **8080** |
| `airflow-scheduler` | DAG scheduling & monitoring |

## Key Design Decisions

- Custom Docker image with `boto3` + `apache-airflow-providers-amazon`
- Scripts mounted via Docker volume: `../scripts:/opt/airflow/scripts`
- Environment variables via `.env` (AWS creds, S3 bucket, Kaggle creds)

# 🔄 DAG 1 — `project_dag_ingest`

## Ingest Pipeline (standalone or triggered)

```
land_raw_dataset → upload_ingest_script → create_emr_cluster
    → wait_for_cluster_ready → submit_spark_step
    → wait_for_step_completion → terminate_emr_cluster
```

| Task | Action |
|---|---|
| `land_raw_dataset` | Kaggle → S3 `/raw/` (skips if data exists) |
| `upload_ingest_script` | Local `ingest.py` → S3 `/scripts/` |
| `create_emr_cluster` | Spin up transient EMR (or reuse from parent) |
| `submit_spark_step` | `spark-submit --deploy-mode cluster` |
| `terminate_emr_cluster` | Clean up (only if DAG owns lifecycle) |

# 🔄 DAG 2 — `big_data_pipeline`

## Main Orchestration DAG

```
create_emr_cluster → wait_for_cluster_ready
    → run_ingest_pipeline → run_merge_pipeline
    → run_aggregate_pipeline → trigger_glue_crawler
    → wait_for_crawler → terminate_emr_cluster
```

## Key Features

- **Shared EMR cluster** — created once, passed to child DAGs via `conf`
- **TriggerDagRunOperator** — triggers child DAGs and waits for completion
- **Lifecycle management** — cluster terminated after all jobs (with `ALL_DONE` trigger rule)
- **Glue Crawler** triggered after all ETL stages complete

# 🗂️ Data Catalog & Query — Glue + Athena

## AWS Glue

- **Crawler:** `bigdata-project-processed-crawler`
- Scans `/processed/` prefix automatically
- Populates **Glue Data Catalog** → `bigdata_project_db`
- Schema auto-discovery from Parquet metadata

## Amazon Athena

- **Workgroup:** `bigdata-project-workgroup`
- Queries via Glue Catalog — no data loading required
- Results stored in `s3://.../athena-results/`
- Pre-configured named query for quick start

# 🛠️ DevOps & Automation

## One-Command Setup

```
./bin/setup.sh      # Deploy everything
./bin/teardown.sh   # Destroy everything
./bin/cost-check.sh # Check AWS spending
```

## `setup.sh` Workflow

1. ✅ Check prerequisites (AWS CLI, Terraform, Docker)
2. 🔑 Prompt for AWS credentials
3. 🪣 Prompt for unique S3 bucket name
4. 🏗️ `terraform init` → `plan` → `apply`
5. ⬆️ Upload PySpark scripts to S3
6. 📝 Generate Airflow `.env`

# 🔒 Security & Best Practices

## Infrastructure

- S3 bucket versioning enabled, public access **blocked**
- IAM roles follow **least-privilege** principle (5 dedicated roles)
- Security groups restrict access by CIDR
- EMR termination protection disabled for cost savings

## Code & Operations

- Sensitive files in `.gitignore` (`terraform/.env`, `airflow/.env`)
- Terraform state management (local + optional S3 remote backend)
- CloudWatch logging for EC2, EMR, and Glue
- Idempotent data landing (skips if raw data already exists)

# 🖥️ Demo Flow

## Step-by-step

1. **Deploy** — `./bin/setup.sh`
2. **Open Airflow** — `http://localhost:8080` (admin/admin)
3. **Trigger** `big_data_pipeline` DAG
4. **Watch** EMR cluster spin up → jobs run → Glue catalog updated
5. **Query** in Athena Console

```sql
SELECT unique_carrier, COUNT(*) AS flights,
       AVG(arr_delay) AS avg_delay
FROM bigdata_project_db.airline_data
GROUP BY unique_carrier
ORDER BY flights DESC;
```

6. **Visualize** in Tableau (connected via Athena)

# 📊 Resources Created

| Service | Resource | Count |
|---|---|---|
| S3 | Versioned bucket | 1 |
| IAM | Roles + Instance Profiles + Policies | 5 / 2 / 5 |
| EC2 | Airflow instance ( `t3.micro` ) | 1 |
| EC2 | Security Groups | 3 |
| EMR | Cluster (1M + 2C + 1T) | 1 |
| Glue | Catalog DB + Crawler | 1 + 1 |
| Athena | Workgroup + Named Query | 1 + 1 |
| CloudWatch | Log Groups | 3 |

# 📚 Lessons Learned

- **Terraform modules** make infrastructure reproducible and composable
- **Transient EMR clusters** save costs vs. long-running clusters
- **Dockerised Airflow** enables local development with production parity
- **Parquet partitioning** dramatically improves query performance in Athena
- **Shared EMR lifecycle** across DAGs reduces cluster spin-up overhead
- **Idempotent tasks** (skip if data exists) prevent duplicate processing

# 🚀 Next Steps

- [ ] **CI/CD** — GitHub Actions for Terraform plan/apply
- [ ] **S3 Remote Backend** — shared Terraform state with DynamoDB locking
- [ ] **Monitoring** — alerting on DAG failures, EMR step errors
- [ ] **Data Quality** — Great Expectations or Deequ validation layer
- [ ] **Incremental Loads** — partition-aware upserts vs. full refresh
- [ ] **Cost Optimization** — EMR Spot instances for Task nodes
- [ ] **Tableau Dashboards** — published workbooks connected via Athena

# 🙏 Thank You

Questions?

`big-data-project`